

V LCD

Contreras R Orlando^{1,2*} and Lara H Kevin^{1,2*}

³Department of Electronic Systems, UAA, Av. Universidad 940,
Aguascalientes, 20131, Aguascalientes, México.

*Corresponding author(s). E-mail(s): [al348390,al464376](mailto:al348390@edu.uaa.mx)@edu.uaa.mx;

Abstract

The objective of this lab practice is develop an API for a LCD Peripheral through C files, this is a huge important way to work because provide us modularity to work with multiple peripherals without getting in with each configuration.

Keywords: ARM, C, API, LCD, Driver,

1 Introduction

Liquid Crystal Displays (LCDs) remain ubiquitous in embedded systems for human-machine interfaces, particularly HD44780-based character displays which offer a cost-effective solution for text-based output. The driver employs careful timing control through precise delay mechanisms and implements the standardized HD44780 command set through bit manipulation operations.

The implementation demonstrates practical solutions to common embedded display challenges, including proper initialization sequencing, atomic character writing through nibble separation, and display control operations. By providing a structured API with clear separation between hardware abstraction and functional operations, this driver serves as a foundation for reliable embedded display systems across various applications, from industrial controls to consumer electronics.

2 Objectives

- Implement the initialization sequence for 4-bit mode.
- Provide functions to write strings and scroll the display left and right.
- Produce a modular, reusable API (LCD handle).
- Verify on hardware and document usage.

3 Method

3.1 Pseudo-code

Here is the pseudo-code that represents the algorithm, it is also included the configuration as masks or

Algorithm 1 LCD 4-Bit Initialization

Input: None

Output: LCD initialized in 4-bit mode

```
1 50 // Wait for LCD power stabilization
2 GPIOC.ODR ← GPIOC.ODR ∧ ¬(1 ≪ 0) // Clear RS bit
3 for i ← 0 to 10 do
4   GPIOF.ODR ← LCD_CONFIG [i] // Load configuration value
5   LCD_Enable() // Pulse enable signal
6 GPIOC.ODR ← GPIOC.ODR ∨ (1 ≪ 0) // Set RS bit for data mode
```

Algorithm 2 LCD Write String Algorithm

Input: String *str* to display

Output: Characters written to LCD

```
7 GPIOC.ODR ← GPIOC.ODR ∨ (1 ≪ 0) // Set RS for data mode
8 for i ← 0 to length(str) − 1 do
9   // Process high nibble
10  GPIOF.ODR ← (str[i] ∧ 0xF0) ≫ 4 // Extract high nibble
11  LCD_Enable() // Write high nibble
12  // Process low nibble
13  GPIOF.ODR ← str[i] ∧ 0x0F // Extract low nibble
14  LCD_Enable() // Write low nibble
```

Algorithm 3 LCD Enable Pulse Generation

Input: None

Output: Enable signal toggled

```
13 GPIOC.ODR ← GPIOC.ODR ⊕ (1 ≪ 2) // Toggle enable bit
14 5 // Wait for signal stabilization
15 GPIOC.ODR ← GPIOC.ODR ⊕ (1 ≪ 2) // Toggle enable bit again
```

Algorithm 4 LCD Clear Display Algorithm

Input: None

Output: LCD display cleared

```
16 GPIOC.ODR ← GPIOC.ODR ∧ ¬(1 ≪ 0) // Clear RS for command mode
17 GPIOF.ODR ← 0x00 // Prepare high nibble
18 LCD_Enable() // Send high nibble
19 GPIOF.ODR ← 0x01 // Clear display command low nibble
20 LCD_Enable() // Send low nibble
21 LCD_Return() // Return home
22 GPIOC.ODR ← GPIOC.ODR ∨ (1 ≪ 0) // Set RS for data mode
```

Algorithm 5 LCD Return Home Algorithm

Input: None**Output:** Cursor returned to home position

```
23 GPIOC.ODR ← GPIOC.ODR ∧ ¬(1 ≪ 0)      // Clear RS for command mode
24 GPIOF.ODR ← 0x00                        // Prepare high nibble
25 LCD_Enable()                            // Send high nibble
26 GPIOF.ODR ← 0x02                        // Return home command low nibble
27 LCD_Enable()                            // Send low nibble
28 GPIOC.ODR ← GPIOC.ODR ∨ (1 ≪ 0)        // Set RS for data mode
```

Algorithm 6 LCD Scroll Left Algorithm

Input: Number of scrolls *scrolls***Output:** LCD display scrolled left

```
29 GPIOC.ODR ← GPIOC.ODR ∧ ¬(1 ≪ 0)      // Clear RS for command mode
30 for i ← 0 to scrolls − 1 do
31   GPIOF.ODR ← 0x01                      // Scroll command high nibble
32   LCD_Enable()                          // Send high nibble
33   GPIOF.ODR ← 0x08                      // Scroll left parameter
34   LCD_Enable()                          // Send low nibble
35 GPIOC.ODR ← GPIOC.ODR ∨ (1 ≪ 0)        // Set RS for data mode
```

Algorithm 7 RCC Configuration Algorithm

Input: None**Output:** RCC configured for GPIO

```
36 RCC.AHB1ENR ← RCC.AHB1ENR ∨ (1 ≪ 5) ∨ (1 ≪ 2)  // Enable GPIOF and
   GPIOC clocks
```

Algorithm 8 GPIO Configuration Algorithm

Input: None**Output:** GPIO pins configured

```
37 GPIOF.MODER ← GPIOF.MODER ∨ (0x55 ≪ 0)  // Set PF0-PF3 as output
38 GPIOF.OSPEEDR ← GPIOF.OSPEEDR ∨ (0xFF ≪ 0) // High speed for
   PF0-PF3
39 GPIOC.MODER ← GPIOC.MODER ∨ (0x11 ≪ 0)  // Set PC0-PC1 as output
```

Algorithm 9 Millisecond Delay Algorithm

Input: Delay value *delay* in milliseconds**Output:** Time delay executed

```
40 for i ← 0 to delay − 1 do
41   for j ← 0 to 11499 do
42     // Inner delay loop - approximately 1ms
```

4 Results and Discussion

4.1 Code

Here is the code that represents the previous pseudo-code

Code: conf.c

```
#include "conf.h"
uint8_t LCD_CONFIG[11] =
{0x03, 0x02, 0X0C,0X02,0X0C,0X00
,0x0F,0x00,0X01,0X00,0x06};
void LCD_Enable(void){
    GPIOC->ODR ^= (1<<2); // enciende ineibol
    delay_ms(5);
    GPIOC->ODR ^= (1<<2); // apaga ineibol
}
void LCD_Clear(void){
    GPIOC->ODR &= (0<<0); // apaga RS
    GPIOF->ODR = (0x00);
    LCD_Enable();
    GPIOF->ODR = (0x01);
    LCD_Enable();
    LCD_Return();
    GPIOC->ODR |= (1<<0); // enciende RS
}

void LCD_Return(void){
    GPIOC->ODR &= (0<<0); // apaga RS
    GPIOF->ODR = (0x00);
    LCD_Enable();
    GPIOF->ODR = (0x02);
    LCD_Enable();
    GPIOC->ODR |= (1<<0); // enciende RS
}

void LCD_TEST(void){
    char tst[9] = "Hola-juan";
    LCD_Init4B();
    LCD_WriteString(tst);
}

void LCD_Init4B(void){
    delay_ms(50);
    GPIOC->ODR &= (1<<0); // apaga RS
    for(uint8_t i=0; i<11; i++){
        GPIOF->ODR = LCD_CONFIG[i];
        LCD_Enable();
    }
    GPIOC->ODR = (1<<0); // enciende RS
}

void LCD_WriteString(char* str){
```

```

    GPIOC->ODR = (1<<0); // enciende RS
    for(uint8_t i=0; i<strlen(str);i++)
    {
        /*Higher nibble*/
        GPIOF->ODR = (str[i] & 0xF0) >> 4;    // high nibble
        LCD_Enable();
        /*Lower nibble*/
        GPIOF->ODR = (str[i] & 0x0F);          // low nibble
        LCD_Enable();
    }
}

void LCD_ScrollLeft(uint8_t scrolls){
    GPIOC->ODR &= (0<<0); // apaga RS
    for(uint8_t i=0; i<scrolls;i++){
        GPIOF->ODR = (0x01);
        LCD_Enable();
        GPIOF->ODR = (0x08);
        LCD_Enable();
    }
    GPIOC->ODR |= (1<<0); // enciende RS
}

void LCD_ScrollRight(uint8_t scrolls){
    GPIOC->ODR &= (0<<0); // apaga RS
    for(uint8_t i=0; i<scrolls;i++){
        GPIOF->ODR = (0x01);
        LCD_Enable();
        GPIOF->ODR = (0x0C);
        LCD_Enable();
    }
    GPIOC->ODR |= (1<<0); // enciende RS
}

}

void confRCC(void){
    RCC->AHB1ENR |= (1<<5)|(1<<2); // GPIOA Y GPIOE
}

void confGPIO(void){
    /*
    A5 Out Push Pull
    */
    GPIOF->MODER |= (0X55<<0);
    GPIOF->OSPEEDR |= (0XFF<<0);

    /*Se configura el PE0 como entrada y PULL UP Externo*/
    GPIOC->MODER |= (0X11<<0);
}

```

```

void delay_ms(uint32_t delay){
    for(uint32_t i=0;i<delay;i++){
        for(uint32_t j=0;j<11500;j++){

        }
    }
}

```

```
Code: conf.c
```

```

#include "conf.h"

int main(void){
    confRCC();
    confGPIO();
    char str1[] = "Hola -DAS";
    LCD_Init4B();
    LCD_WriteString(str1);
    while(1){
    }
}

```

4.2 Wiring Diagram

Here is our circuit implementation; we use the port F as LCD DATA from pin 0:3, the control pins are C0 and C2. [Fig 1]

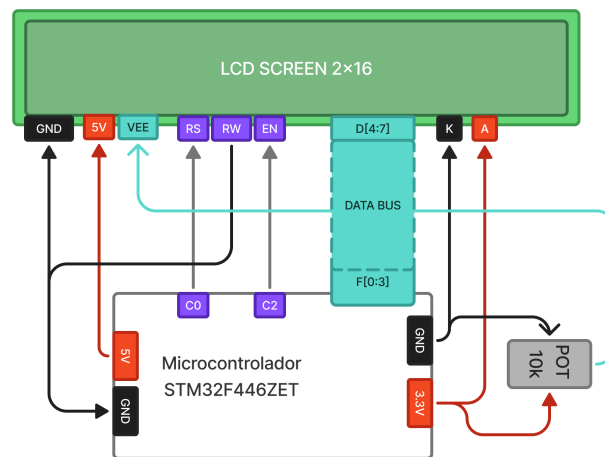


Fig. 1: Wiring Diagram

Q1: Explain the initialization 0x3,0x3,0x3,0x2 sequence. .

```
0x30 Function set 4 bit mode
0x2C Function set 4 bit mode, 2 line, 5x8 dots
0x2C Function set 4 bit mode, 2 line, 5x8 dots
0x0F Display on, cursor on, blinking on
0x01 Clear display
0x06 Entry mode set, increment cursor, no shift
```

Q2: Why tie RW to GND or why not? Tradeoffs.

Because we use LCD only to send data, we are not interested in reading in the internal RAM.

Q3: How would you generalize to 20×4 displays?

The same code works for 4x20, there is no change.

Q4: How to avoid message interleaving when multiple tasks print?

Using a mutex or semaphore to avoid overlapping functions, that assures that each task completes its entire message without being interrupted

5 Conclusion

In this practice, we could observe the functionality of a LCD library to configure, using commands like return home, clear display and print any data. This is useful in a lot of cases when we need show data to users or ourselves in testing proves etc. The creation of a library is a very useful tool when we need to implement a new peripheral, this avoid us to configure it each time and gave us a Hardware Abstraction perspective